Running a 125-Billion-Parameter Model on a Normal PC

Strata: a CUDA and CPU inference engine that serves Qwen3.8-Flash-Next from one 12 GB graphics card, 64 GB of RAM and a six-core desktop processor · September 2026

Abstract. Qwen3.8-Flash-Next is a 125-billion-parameter mixture-of-experts model that activates only 6 billion parameters per token[1–3]. Its smallest good-quality quantization is a 66 GB file[4], five and a half times the memory of a 12 GB graphics card. Strata is an inference engine written for this one model: it keeps the small, always-used part of the network on the GPU, keeps all 24,576 experts in system RAM, computes the experts the GPU does not hold on the CPU in the same instant, and uses the model's own multi-token-prediction layer to verify several tokens per pass. On an RTX 5070 (12 GB), a Ryzen 5 7600 and 64 GB of DDR5, it generates 95 tokens per second at a 4K context with the 2-bit Q2_0 file, 45–95 tokens per second across all three quantizations and contexts from 1K to the full 262K, and reads prompts at 285–571 tokens per second. This paper explains how the engine works, what limits it, what we measured and what could make it faster, in plain language wherever possible.

1 The problem in one paragraph

A large language model produces text one token (roughly one word piece) at a time, and for every token the hardware has to read the model's weights that the token uses. On a normal PC the fastest memory is the graphics card's VRAM (hundreds of GB/s), the next is system RAM (tens of GB/s) and the slowest is the SSD. A 66 GB model cannot live in a 12 GB card. The usual answer, running the rest from system RAM on the CPU with a general-purpose engine such as llama.cpp[9], works but leaves most of the machine idle most of the time. Strata's answer is to specialize: to write the engine around the exact shape of this model and this kind of PC, so that the GPU, the CPU, the RAM and the PCIe link are all busy with the work each is best at, at the same time.

2 Why this model fits a small GPU at all

The model's designers priced every architectural choice in bytes moved per token[1], and it shows. Of its 48 layers, 36 use Gated DeltaNet[10], a recurrent mixer whose memory is a fixed-size state instead of a growing cache; the other 12 use Qwen Sparse Attention, which has only two key/value heads and attends to at most about 2,048 selected positions no matter how long the context is[1, 21]. Every layer ends in a mixture of 512 experts of which the router picks 10, so a token touches only 2% of the expert weights. A 51-billion-parameter n-gram embedding table is addressed by the last three token ids, so its rows are known before they are needed. Table 1 is the byte census of the 2-bit file.

在普通 PC 上运行 1250 亿参数模型

Strata：一个 CUDA 与 CPU 推理引擎，仅凭一块 12 GB 显卡、64 GB 内存和一颗六核桌面处理器即可运行 Qwen3.8-Flash-Next · 2026 年 9 月

摘要。Qwen3.8-Flash-Next 是一个 1250 亿参数的混合专家模型，每个 token 仅激活 60 亿个参数[1–3]。其质量可用的最小量化版本是一个 66 GB 的文件[4]，，相当于一块 12 GB 显卡内存的五倍半。Strata 是专为这一模型编写的推理引擎：它将网络中始终使用的小部分保留在 GPU 上，将所有 24,576 个专家保留在系统内存中，同时由 CPU 计算 GPU 未持有的专家，并利用模型自带的多 token 预测层在每次前向中验证多个 token。在 RTX 5070（12 GB）、Ryzen 5 7600 和 64 GB DDR5 上，使用 2 位 Q2_0 文件、4K 上下文时，它每秒生成 95 个 token；在全部三种量化以及从 1K 到完整 262K 的上下文中，每秒生成 45–95 个 token；读取提示词的速度为每秒 285–571 个 token。本文尽可能用通俗的语言解释该引擎如何工作、受什么限制、我们测量了什么，以及哪些改进可以让它更快。

1 用一段话说明问题

大语言模型一次生成一个 token（大致相当于一个词片），而每生成一个 token，硬件都必须读取该 token 所用到的那部分模型权重。在普通 PC 上，最快的存储器是显卡的 VRAM（数百 GB/s），其次是系统 RAM（数十 GB/s），最慢的是 SSD。一个 66 GB 的模型无法完全放进 12 GB 的显卡中。常见做法是借助 llama.cpp[9]，这类通用引擎，把剩余部分放在系统 RAM 中由 CPU 运行；这样做可行，但会让机器的大部分时间处于闲置状态。Strata 的答案是专门化：围绕这个模型和这类 PC 的确切形态来编写引擎，让 GPU、CPU、RAM 和 PCIe 链路同时忙于各自最擅长的工作。

2 这个模型为何能适配小显存 GPU

该模型的设计者按每个 token[1]，所移动的字节数来衡量每一项架构选择，而结果也印证了这一点。在其 48 层中，有 36 层使用 Gated DeltaNet[10]，——一种循环混合器，其内存是固定大小的状态，而不是不断增长的缓存；其余 12 层使用 Qwen Sparse Attention，它只有两个键/值头，并且无论上下文多长[1, 21]，都最多只关注约 2,048 个选定位置。每一层最后都接入一个由 512 个专家组成的混合体，路由器会从中选择 10 个，因此一个 token 只触及 2% 的专家权重。一个 510 亿参数的 n-gram 嵌入表由最后三个 token ID 寻址，因此其行在被需要之前就已知晓。表 1 是 2-bit 文件的字节统计。

Part of the model	Stored (Q2_0 file)	Read per token	Where Strata keeps it
Routed experts (48 layers × 512)	34.0 GB	0.66 GB (2%)	**RAM** (all) + **VRAM** cache (the most-used ~4,500)
Mixers, attention, shared experts, routers	1.8 GB	1.8 GB	**VRAM**
Gated residual (hyper-connections)	1.3 GB	1.3 GB	**VRAM**
Output head	0.44 GB	0.44 GB	**VRAM**
N-gram embedding table	28.8 GB	≤ 23 KB	**SSD** + OS page cache
Multi-token-prediction (MTP) layer	0.8 GB (from the BF16 checkpoint)	per draft	**VRAM**

Table 1. Where the bytes are. The GPU-resident part is about 3.5 GB; the expert traffic, 0.66 GB per token, is the quantity the design is organized around[4, 9].

Figure 1. The three memory tiers and what Strata places in each (RTX 5070 12 GB, 64 GB DDR5, PCIe 4.0 x16, NVMe SSD). Bandwidths are measured on the test machine.

3 How Strata works

3.1 Three memory tiers

VRAM holds everything that every token needs: the attention and DeltaNet mixers, the gated-residual weights, the routers, the shared experts, the output head, the MTP draft layer, the key/value cache and the recurrent state, and an *expert cache* that fills whatever VRAM is left. **System RAM** holds all 24,576 experts in one pinned (page-locked) arena, so the CPU can compute any of them in place and the GPU can pull any of them over PCIe by DMA. The **SSD** keeps the 28.8 GB n-gram table; the engine reads the 16 rows each token needs through the operating system's page cache, one step ahead of the layer that uses them.

3.2 One layer, three workers at once

For every layer the GPU runs the mixer and the router and writes the ten chosen expert ids into a small block of pinned host memory (a “doorbell”). The CPU, which spins on that doorbell instead of waiting on the driver, immediately splits the experts: the ones already in the VRAM cache go to the GPU, a share of the others is copied to the GPU over PCIe by its copy engine while everything else runs (55% of the misses for the i-quant files, 20% for Q2_0, whose CPU needs the RAM bandwidth itself), and the rest are computed by the CPU's cores next to the RAM. The CPU writes its results back into mapped memory, the GPU adds

模型部分	存储（Q2_0 文件）	读取每个词元	Strata 将其保存在何处
路由专家（48 层 × 512）	34.0 GB	0.66 GB（2%）	RAM（全部）+ VRAM 缓存（最常用的 ~4,500）
混合器、注意力、共享专家，路由器	1.8 GB	1.8 GB	**VRAM**
门控残差（超连接）	1.3 GB	1.3 GB	**VRAM**
输出头	0.44 GB	0.44 GB	**VRAM**
N-gram 嵌入表	28.8 GB	≤ 23 KB	SSD + 操作系统页面缓存
多 token 预测（MTP）层	0.8 GB（来自 BF16 检查点）	每个草稿	**VRAM**

表 1. 字节都去了哪里。驻留在 GPU 上的部分约为 3.5 GB；专家流量为每 token 0.66 GB，而设计正是围绕这一数量来组织的[4, 9]。

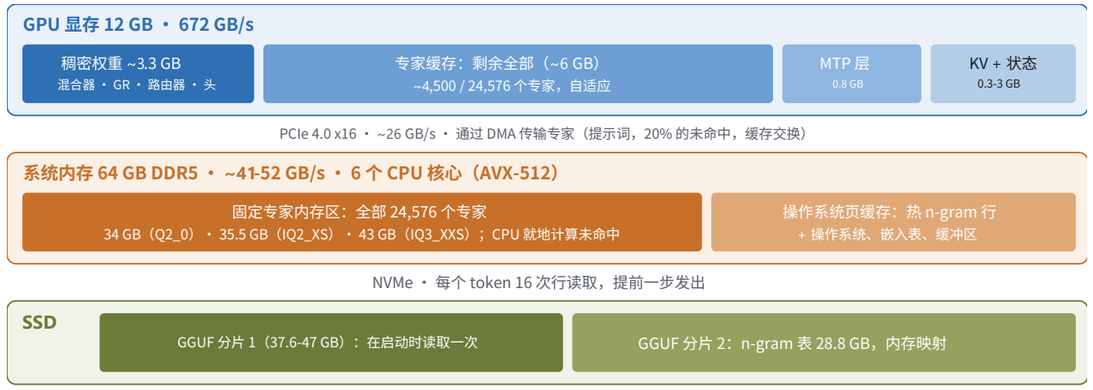

图 1. 三个 me 内存层级以及 Strata 在各层中放置的内容（RTX 5070 12 GB、64 GB DDR5、PCIe 4.0 x16、NVMe SSD）。带宽 ths 是在测 试机器上测量的。

3 Strata 如何运作

3.1 三种内存层级

VRAM 保存每个 token 所需的一切：注意力混合器与 DeltaNet 混合器、门控残差权重、路由器、共享专家、输出头、MTP 草稿层、键/值缓存和循环状态，以及一个用于填充剩余 VRAM 的专家缓存。系统内存将所有 24,576 个专家保存在同一个固定（页锁定）的内存区域中，因此 CPU 可以就地计算其中任意一个专家，GPU 也可以通过 DMA 经由 PCIe 拉取其中任意一个专家。SSD 保存着 28.8 GB 的 n-gram 表；引擎通过操作系统的页缓存读取每个 token 所需的 16 行数据，并比使用这些数据的层提前一步完成读取。

3.2 一层，三个工作单元同时工作

对于每一层，GPU 会运行混合器和路由器，并把选出的十个专家 ID 写入一小块锁页主机内存（一个“门铃”）。CPU 会在该门铃上自旋，而不是等待驱动程序；它会立即拆分这些专家：已位于 VRAM 缓存中的专家交给 GPU；其余专家中的一部分由 GPU 的复制引擎通过 PCIe 复制到 GPU，同时其他工作继续运行（对于 i-quant 文件，未命中项中有 55% 如此复制；对于 Q2_0 为 20%，因为其 CPU 自身需要 RAM 带宽），剩下的则由紧邻 RAM 的 CPU 核心计算。CPU 将其结果写回映射内存，GPU 再加上

everything up, and the next layer starts. The whole 48-layer pass is one captured CUDA graph per window size, so the host never waits on a synchronizing driver call.

Figure 2. One layer of a verify window. The GPU's resident experts and the CPU's missed experts run at the same time; the next layer waits for the slower of the two.

3.3 Checking several tokens per pass: speculative decoding with the model's own MTP layer

Reading 3.5 GB of dense weights for every single token is the GPU's main cost, and it barely grows when four tokens are processed together. Qwen3.8-Flash-Next ships a one-layer multi-token-prediction (MTP) head[1, 12] that guesses the next tokens cheaply. The GGUF files do not include it, so Strata fetches only its tensors (5 GB) from the original BF16 checkpoint[3] and keeps it in VRAM with its experts in 2-bit form and a reduced output head over 40,525 likely tokens[7, 8]. Each round, the last accepted token and up to three drafts go through all 48 layers as one *verify window*; the engine keeps every draft the full model agrees with and one more token from the model itself[11]. A draft only enters the window while the MTP layer is at least 50% confident. The result is exact: the text is identical, token for token, to plain greedy decoding, which we test with forced wrong drafts and rollbacks.

3.4 An expert cache that follows the conversation

Routing is uneven: some experts are chosen far more often than others. At startup Strata fills the VRAM cache with the most-used experts from a profile recorded on other prompts; every four rounds it swaps up to 96 experts that the current conversation keeps asking for into the slots of the least-used ones, while the GPU is drafting. Figure 3 shows how the share of experts served from VRAM (the hit rate) grows with the number of experts that fit.

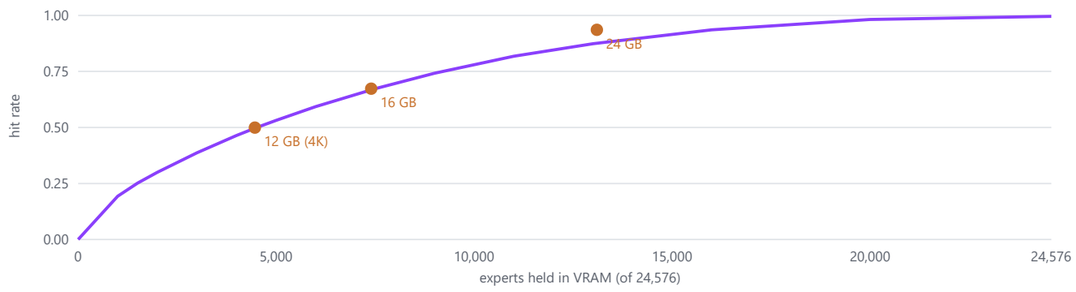

Figure 3. Hit rate of a profile-filled expert cache against its size, measured on held-out prompts (each prompt scored against a profile built from the other nine). The adaptive swapping adds to this: on the 12 GB card the engine measures 0.72 at 4K context instead of the curve's 0.50.

3.5 Reading the prompt: streaming experts to the GPU

Prompts are processed in chunks of 2,048 tokens. With that many tokens every expert is used many times, so the right place to compute is the GPU: for each layer, the experts that are not cached are copied from the pinned arena over PCIe while the GPU computes the previous ones, dequantized to half precision and

把所有内容相加起来，然后下一层开始。整个 48 层的一遍处理，对每种窗口大小都是一个已捕获的 CUDA 图，因此主机从不会等待一次会引发同步的驱动程序调用。

图 2. 验证窗口的一层。GPU 常驻专家与 CPU 未命中专家同时运行；下一层会等待两者中较慢的一方。

3.3 每次检查多个 token：使用模型自带 MTP 层的投机解码

每处理一个 token 都要读取 3.5 GB 的稠密权重，这是 GPU 的主要开销，而同时处理四个 token 时，这一开销几乎不会增加。Qwen3.8-Flash-Next 自带一个单层多 token 预测（MTP）头[1,12]，可以低成本地猜测接下来的 token。GGUF 文件并不包含它，因此 Strata 只从原始 BF16 checkpoint 中获取它的张量（5 GB）[3]，并将其与以 2-bit 形式存放的专家以及一个缩减后的、覆盖 40,525 个可能 token 的输出头一起保留在 VRAM 中[7, 8]。每一轮中，最后一个被接受的 token 以及最多三个草稿会作为一个验证窗口穿过全部 48 层；引擎会保留完整模型认可的每一个草稿，并额外保留一个由模型自身给出的 token[11]。只有当 MTP 层至少有 50% 的置信度时，草稿才会进入该窗口。结果是精确的：生成的文本与普通贪心解码逐 token 完全一致，我们通过强制错误草稿和回滚对此进行了测试。

3.4 跟随对话的专家缓存

路由并不均衡：某些专家被选中的频率远高于其他专家。启动时，Strata 会根据在其他提示词上记录的使用画像，用最常用的专家填充 VRAM 缓存；此后每四轮，它会在 GPU 起草期间，将当前对话持续请求的最多 96 个专家换入最不常用专家的槽位。图 3 展示了从 VRAM 提供的专家占比（命中率）如何随可容纳的专家数量增长。

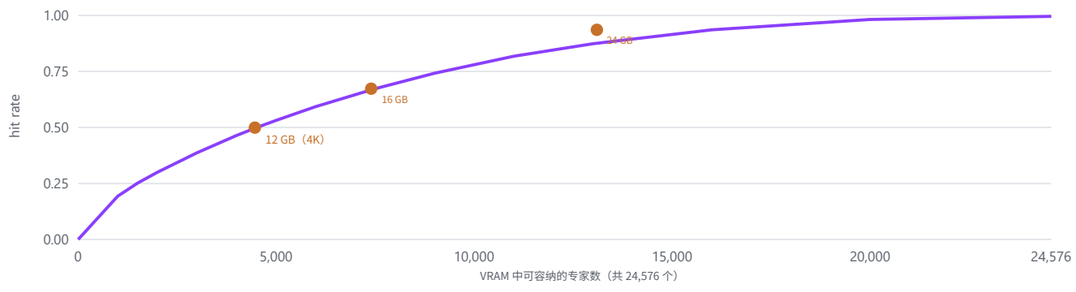

图 3. 按配置填充的专家缓存的命中率随其大小的变化，在留出提示词上测得（每个提示词都根据由其余九个构建的配置进行评分）。自适应交换进一步提升了这一结果：在 12 GB 显卡上，引擎在 4K 上下文下测得 0.72，而不是曲线上的 0.50。

3.5 读取提示词：将专家流式传输到 GPU

提示按 2,048 个 token 为一组进行处理。在这么多 token 的情况下，每个专家都会被多次使用，因此合适的计算位置是 GPU：对于每一层，未缓存的专家会在 GPU 计算前一批专家的同时，通过 PCIe 从固定内存区复制过来，反量化为半精度并

multiplied with cuBLAS[18]. The buffers are borrowed from the expert cache and refilled afterwards. Prompt speed stays flat from 4K to 262K tokens because the sparse attention reads a bounded set of positions.

3.6 Three quantizations of the same model

ISTA-DASLab publishes three GSQ-RCO quantizations[4]: **Q2_0** (2.25 bits per expert weight, the fastest), **IQ2_XS** and **IQ3_XXS** (llama.cpp's "i-quants", which encode groups of eight weights as an index into a codebook of lattice points[9]). Q2_0 is a plain grid {-1, 0, 1, 2} × scale, which suits an integer dot-product instruction; Strata repacks its experts once and computes them with a hand-written AVX-512 VNNI kernel that runs near the RAM's bandwidth. The i-quants cannot be repacked into that form without losing information, so for them Strata keeps every tensor exactly as the GGUF stores it: the GPU uses dot products and dequantizers transcribed from llama.cpp's CUDA code, the CPU uses ggml's own vector dot products, and a new AVX-512 kernel handles the Q2_0 down projections and the experts that several tokens of a window share. Each of these was checked against llama.cpp's reference arithmetic on real layers of the files.

3.7 Long context

Past 8K tokens the key/value cache is stored in 8-bit integers. The MTP layer attends to the last 32,768 positions, which keeps drafting cheap at 262K. The cost of a long context is mainly VRAM: at 262K the key/value cache of the 12 sparse attention layers, their indexer keys and the larger prompt buffers take about 4 GB, so the automatic sizing fits about 1,600 experts on the GPU instead of about 4,500.

与 cuBLAS[18] 相乘。缓冲区从专家缓存中借用，之后会重新填充。提示处理速度在 4K 到 262K 个 token 范围内保持平稳，因为稀疏注意力只读取有限的一组位置。

3.6 同一模型的三种量化

ISTA-DASLab 发布了三种 GSQ-RCO 量化[4]：Q**2_0**（每个专家权重 2.25 比特，速度最快）、IQ**2**_XS 和 IQ**3**_XXS（llama.cpp 的“i-quants”，它们将每八个权重编码为格点码本中的索引[9]）。Q2_0 是一种普通网格 {-1, 0, 1, 2} × 缩放，适合整数点积指令；Strata 将其专家重新打包一次，并用一个手写的 AVX-512 VNNI 内核计算它们，该内核运行速度接近 RAM 的带宽。i-quants 无法在不丢失信息的情况下重新打包成那种形式，因此对于它们，Strata 让每个张量完全保持 GGUF 存储它的样子：GPU 使用从 llama.cpp 的 CUDA 代码转写而来的点积和反量化器，CPU 使用 ggml 自带的向量点积，而一个新的 AVX-512 内核处理 Q2_0 down 投影以及一个窗口内多个 token 共享的专家。上述每一项都在文件的真实层上对照 llama.cpp 的参考算术进行了检查。

3.7 长上下文

超过 8K 个 token 后，键/值缓存以 8 位整数存储。MTP 层会关注最后 32,768 个位置，这使 262K 上下文下的草稿生成开销保持低廉。长上下文的成本主要来自显存：在 262K 时，12 个稀疏注意力层的键/值缓存、它们的索引器键以及更大的提示词缓冲区大约占用 4 GB，因此自动容量配置可在 GPU 上容纳约 1,600 个专家，而不是约 4,500 个。

4 Results

Machine: RTX 5070 12 GB (PCIe 4.0 x16, driving the display), AMD Ryzen 5 7600 (6 cores, AVX-512), 64 GB DDR5-5200 (EXPO off), NVMe SSD, Windows 10. **Method:** one code-agent prompt per length (a real code-review task over long source files), 256 generated tokens, greedy decoding, MTP speculation on (up to 4 tokens per window, drafts kept while their probability is at least 0.5), expert cache sized automatically from free VRAM, 8-bit KV cache from 32K up. The expert profile was recorded on different prompts. Every number is one full run of the engine from a cold start of the prompt; prompt speed includes all 48 layers and the n-gram lookups. Q2_0 runs from its repacked (canonical) pack, IQ2_XS and IQ3_XXS from native packs (section 3.6).

Prompt processing (tokens per second)

Model	1K	4K	32K	64K	128K	262K
Q2_0	389	539	571	561	543	496
IQ2_XS	332	463	495	486	472	437
IQ3_XXS	285	410	435	427	414	–

Table 2. Prompt processing speed, measured. Prompt lengths: 1,017 / 3,562 / 31,566 / 64,162 / 128,478 / 259,943 tokens (the last is the model's full 262,144-token window). IQ3_XXS at 262K was not measured: with its 43 GB expert arena, a 260K-token context and the operating system, the run brings a 64 GB machine to its memory limit, and it was stopped there. Treat 128K as the practical ceiling for IQ3_XXS on 64 GB.

Output (tokens per second)

Model	1K	4K	32K	64K	128K	262K
Q2_0	88.7	94.6	87.5	76.4	65.1	56.3
IQ2_XS	82.0	78.0	65.3	63.7	52.0	48.0
IQ3_XXS	64.6	65.6	57.3	54.4	44.8	–

Table 3. Generation speed with MTP speculative decoding, measured over 256 generated tokens.

Details

Run	1K	4K	32K	64K	128K	262K
Q2_0: time to first token (s)	2.7	6.7	55.3	114.4	236.7	523.8
Q2_0: tokens per verify window	2.71	3.23	3.11	2.80	2.65	2.46
Q2_0: experts in VRAM	4,517	4,464	4,174	3,809	3,079	1,588
IQ2_XS: time to first token (s)	3.1	7.8	63.8	132.0	272.5	595.1
IQ2_XS: tokens per verify window	3.25	3.28	2.53	2.72	2.44	2.42
IQ2_XS: experts in VRAM	4,621	4,568	4,289	3,936	3,229	1,796
IQ3_XXS: time to first token (s)	3.7	8.8	72.7	150.2	310.1	–
IQ3_XXS: tokens per verify window	3.63	3.24	2.78	2.88	2.42	–
IQ3_XXS: experts in VRAM	3,556	3,511	3,279	2,990	2,407	–

Table 4. Time to first token, tokens produced per verify window (accepted drafts + 1) and the number of experts the automatic sizing fitted into VRAM.

4 结果

机器：RTX 5070 12 GB（PCIe 4.0 x16，负责驱动显示器）、AMD Ryzen 5 7600（6 核，AVX-512）、64 GB DDR5-5200（EXPO 关闭）、NVMe SSD、Windows 10。方法：每个长度使用一个代码智能体提示词（对长源文件执行真实的代码审查任务），生成 256 个 token，贪婪解码，开启 MTP 推测（每个窗口最多 4 个 token，保留概率至少为 0.5 的草稿），专家缓存根据空闲显存自动调整大小，从 32K 起使用 8 位 KV 缓存。专家配置是在不同提示词上记录的。每个数字都是引擎从提示词冷启动开始完整运行一次的结果；提示词速度包含全部 48 层以及 n-gram 查找。Q2_0 从其重新打包的（规范）包运行，IQ2_XS 和 IQ3_XXS 从原生包运行（第 3.6 节）。

提示处理（token/秒）

模型	1K	4K	32K	64K	128K	262K
Q2_0	389	539	571	561	543	496
IQ2_XS	332	463	495	486	472	437
IQ3_XXS	285	410	435	427	414	–

表 2. 提示处理速度实测值。提示长度：1,017 / 3,562 / 31,566 / 64,162 / 128,478 / 259,943 个 token（最后一个是模型完整的 262,144 token 窗口）。IQ3_XXS 在 262K 下未测量：其 43 GB 的专家内存区、260K token 上下文以及操作系统，会使这次运行将 64 GB 机器推到内存极限，因此在此停止。对于 64 GB 上的 IQ3_XXS，应将 128K 视为实际上的上限。

输出（token/秒）

模型	1K	4K	32K	64K	128K	262K
Q2_0	88.7	94.6	87.5	76.4	65.1	56.3
IQ2_XS	82.0	78.0	65.3	63.7	52.0	48.0
IQ3_XXS	64.6	65.6	57.3	54.4	44.8	–

表 3. 使用 MTP 推测解码的生成速度，在 256 个生成 token 上测得。

详情

Run	1K	4K	32K	64K	128K	262K
Q2_0：首个 token 的时间（秒）	2.7	6.7	55.3	114.4	236.7	523.8
Q2_0：每个验证窗口的 token 数	2.71	3.23	3.11	2.80	2.65	2.46
Q2_0：VRAM 中的专家数	4,517	4,464	4,174	3,809	3,079	1,588
IQ2_XS：首个 token 的耗时（秒）	3.1	7.8	63.8	132.0	272.5	595.1
IQ2_XS：每个验证窗口的 token 数	3.25	3.28	2.53	2.72	2.44	2.42
IQ2_XS：VRAM 中的专家	4,621	4,568	4,289	3,936	3,229	1,796
IQ3_XXS：首个 token 时间（秒）	3.7	8.8	72.7	150.2	310.1	–
IQ3_XXS：每个验证窗口的 token 数	3.63	3.24	2.78	2.88	2.42	–
IQ3_XXS：显存中的专家数	3,556	3,511	3,279	2,990	2,407	–

表 4. 首个 token 生成时间、每个验证窗口生成的 token 数（接受的草稿 + 1）以及自动尺寸调整适配进 VRAM 的专家数量。

5 Where the time goes

A decode round (one verify window of up to four tokens) breaks down as in Table 5. “GPU” is the time the CPU waits for the GPU's per-layer work (weights read once per window, attention, the cached experts); “CPU experts” is the time the GPU waits for the CPU to finish the experts that were not in VRAM.

Model (4K)	round ms	GPU	CPU experts	drafting	other	tokens/round	VRAM hit rate	CPU GB/s
Q2_0	34.3	14.6	13.4	2.0	4.3	3.23	0.72	41
IQ2_XS	42.0	14.7	21.1	2.2	4.0	3.28	0.78	20
IQ3_XXS	49.4	15.3	27.1	2.1	4.9	3.24	0.71	24

Table 5. One verify window at 4K context, in milliseconds (measured). “CPU GB/s” is the rate at which the CPU streams expert weights while computing.

The single biggest bottleneck, in plain words.

- **CPU:** for the 2-bit Q2_0 file the CPU is not the limit, the RAM behind it is: the expert kernel already streams weights at about 42 GB/s, close to what the memory delivers while the GPU is also pulling data. For the i-quant files the CPU *itself* is the limit: decoding a codebook weight costs several instructions (table lookups, sign masks), and six cores manage only about 25 GB/s. That is why IQ3_XXS is slower even though it is only 25% bigger.
- **RAM:** bandwidth (every missed expert is read from it each round) and capacity. 64 GB is just enough: the expert arena alone is 34 GB (Q2_0), 35.5 GB (IQ2_XS) or 43 GB (IQ3_XXS). Running the RAM at its rated EXPO speed instead of the default 5200 MT/s would speed up the CPU half directly. On a DDR4 system (roughly half the bandwidth of DDR5) the CPU half of a Q2_0 round takes about twice as long; the i-quants, which are limited by arithmetic rather than bandwidth, lose less. Either way, a bigger GPU cache is the best compensation.
- **GPU:** its *size* matters more than its speed. Every extra gigabyte of VRAM holds about 700 more experts, and every expert in VRAM is one the CPU does not have to compute. The GPU's own work (about 14 ms per window, mostly reading 3.5 GB of dense weights) is the second limit, and it is set by memory bandwidth.

Put together: on this machine the GPU and the CPU each need about half of every round, and they wait for each other at every layer. The engine is balanced, which is good, but it means that no single component upgrade doubles the speed.

6 Findings

1. **The engine is balanced, so both halves matter.** At 4K context with Q2_0 a verify window takes about 33 ms: the CPU waits about 14 ms for the GPU and the GPU about 13 ms for the CPU. Speeding up only one side cannot help by more than about a third.
2. **Speculation with the model's own MTP layer is worth 1.6–1.8×.** Plain decoding reached 47–57 tokens per second at 4K; with the verify window it is 82–92. That is less than the 2.5–3× engines report when the whole model is in VRAM[7, 8], because every extra token in the window routes to its own CPU experts, but more than the 1.1–1.7× the original design estimated.
3. **Exact is achievable and worth insisting on.** The speculative output is identical to plain greedy decoding token for token, including with deliberately wrong drafts. Against llama.cpp the mean KL divergence of the next-token distribution is 0.022, below the 0.058 that llama.cpp's own CPU and GPU builds differ by[9].

5 时间花在哪里

一次解码轮次（一个最多包含四个 token 的验证窗口）的耗时分解如表 5 所示。“GPU”指 CPU 等待 GPU 完成逐层工作的时间（每个窗口读取一次权重、注意力计算、已缓存的专家）；“CPU 专家”指 GPU 等待 CPU 完成不在 VRAM 中的专家计算的时间。

模型（4K）	轮次 ms	GPU	CPU 专家	生成草稿	其他	tokens/轮	显存命中率	CPU GB/s
Q2_0	34.3	14.6	13.4	2.0	4.3	3.23	0.72	41
IQ2_XS	42.0	14.7	21.1	2.2	4.0	3.28	0.78	20
IQ3_XXS	49.4	15.3	27.1	2.1	4.9	3.24	0.71	24

表 5. 4K 上下文下的一次验证窗口，单位为毫秒（实测）。“CPU GB/s” 是 CPU 在计算时流式传输专家权重的速率。

用直白的话说，最大的单一瓶颈。

- CPU：对于 2-bit Q2_0 文件，瓶颈不在 CPU，而在其背后的 RAM：专家内核已经以约 42 GB/s 的速度流式传输权重，接近内存在 GPU 同时取数时所能提供的带宽。对于 i-quant 文件，瓶颈就在 CPU 本身：解码一个码本权重需要多条指令（查表、符号掩码），六个核心只能达到约 25 GB/s。这就是为什么 IQ3_XXS 虽然只大了 25%，却更慢。

- RAM：带宽（每轮每个未命中的专家都要从内存读取）和容量。64 GB 勉强够用：仅专家区就占 34 GB（Q2_0）、35.5 GB（IQ2_XS）或 43 GB（IQ3_XXS）。让 RAM 以标称 EXPO 速度而不是默认 5200 MT/s 运行，会直接加快 CPU 负责的那一半。在 DDR4 系统上（带宽约为 DDR5 的一半），Q2_0 一轮中 CPU 负责的那一半耗时约翻倍；而 i-quant 受限于算术性能而非带宽，损失较小。无论哪种情况，更大的 GPU 缓存都是最好的补偿。

- GPU：其容量比速度更重要。每多 1 GB 显存，大约能多容纳 700 个专家，而显存中的每个专家都是 CPU 不必计算的。GPU 自身的工作（每个窗口约 14 ms，主要是读取 3.5 GB 稠密权重）是第二重限制，而它由内存带宽决定。

综合来看：在这台机器上，GPU 和 CPU 各自需要每一轮约一半的时间，并且它们在每一层都要互相等待。该引擎是均衡的，这是好事，但这也意味着没有任何单一组件的升级能让速度翻倍。

6 项发现

1. 引擎是均衡的，因此两半都很重要。在 4K 上下文下，使用 Q2_0 a 验证窗口约需 33 ms：CPU 等待 GPU 约 14 ms，GPU 等待 CPU 约 13 ms。只加速一侧最多只能带来约三分之一的提升。

2. 使用模型自带的 MTP 层进行推测，价值为 1.6–1.**8**×。在 4K 下，普通解码达到每秒 47–57 个 token；使用验证窗口后为 82–92。这低于整个模型都在 VRAM[7, 8]，中时各引擎报告的 2.5–3× ，因为窗口中的每个额外 token 都会路由到各自对应的 CPU 专家；但高于原始设计估算的 1.1–1.7× 。

3. 精确性是可以实现的，而且值得坚持。推测输出与普通贪心解码逐 token 完全一致，即使故意使用错误的草稿也是如此。与 llama.cpp 相比，下一 token 分布的平均 KL 散度为 0.022，低于 llama.cpp 自身 CPU 与 GPU 构建之间相差的 0.058[9]。

4. **An adaptive expert cache beats a static one by a wide margin.** A profile recorded on other prompts serves 50% of the routed experts from 4,500 VRAM slots; swapping in the conversation's own hot experts raises that to about 72%.
5. **Overlapping the two halves inside a layer did not pay.** Splitting each window into two token groups, so that one group's CPU experts run while the GPU processes the other group, is exact but about 7% slower (85.8 vs 91.8 tokens per second at 4K): the dense weights are read twice and each group shares fewer experts across its tokens.
6. **The biggest CPU gain came from the instruction mix.** Rewriting the Q2_0 expert kernel for 512-bit AVX-512 (one unpack, one VNNI dot product per 64 weights) brought the CPU to about 42 GB/s of expert weights, close to what the RAM delivers while the GPU is also reading from it (41 GB/s concurrent, 52 GB/s alone).
7. **The i-quants are limited by CPU arithmetic, not by RAM.** Decoding a codebook weight costs table lookups and sign handling; both ggml's AVX2 kernels and our AVX-512 ones run at about 5 GB/s per core, 23–26 GB/s on six cores. Decoding once for several tokens helps (2.0–2.4× at three tokens), but most experts serve a single token. ggml's CPU backend has no SIMD kernel for Q2_0 on x86 at all; the IQ files' Q2_0 down projections needed their own (7× faster than the scalar one).
8. **Size the cache in bytes, not in slots.** IQ3_XXS experts range from 1.3 to 2.3 MB depending on the layer. Sizing each VRAM slot to its layer instead of to the largest expert fits 3,556 experts instead of 2,673 and raised decode by 13% at 1K.
9. **The overlap that paid: the copy engine.** A share of the missed experts can be read by the GPU straight from RAM over PCIe. Done by a copy kernel, those reads sat on the GPU's critical path, and more than 20% of the misses made decoding slower. Issuing them instead as DMA copies from the CPU thread the moment it plans the layer lets the GPU's copy engine run beside both the CPU's experts and the GPU's own work; the GPU computes its cached experts first and the copied ones when they land. For the i-quant files, whose CPU is limited by arithmetic and leaves RAM bandwidth free, 55% of the misses now go over PCIe: IQ2_XS at 4K went from 70 to 78 tokens per second and IQ3_XXS from 56 to 65. Q2_0 keeps the copy kernel at 20%: its CPU already uses the RAM's bandwidth, so DMA would only take bandwidth away from it.
10. **Refill the cache without making anyone wait.** The adaptive cache used to copy up to 96 experts (130–200 MB) into VRAM between rounds, and the next round waited for the copies. Evicting the old expert at once and admitting the new one only when its copy has landed removed the wait: Q2_0 at 4K went from 91.7 to 94.4 tokens per second.
11. **Prompt speed is flat with context length.** 539 tokens per second at 4K and 496 at 262K: the sparse attention reads a bounded set of positions. It is bound by per-expert kernel launches and half-precision dequantization, using only about 7 GB/s of the 26 GB/s PCIe link.
12. **Long context costs VRAM more than compute.** At 262K tokens the 8-bit KV cache leaves room for about 1,600 experts instead of 4,500, and the MTP layer's drafts are accepted less often; together they take Q2_0 decoding from about 95 to about 56 tokens per second.
13. **Windows needs care with pinned memory.** Page-locking 34–43 GB as one range fails on Windows; per-layer ranges plus a working-set lock for the remainder work, and a PCIe read of a missed expert must never span two ranges.

7 What could make it faster

- **Keep the conversation's state between requests.** Today every request processes its whole prompt again. Saving the recurrent state at the end of each turn (a few hundred MB) and the KV cache would

4. 自适应专家缓存以很大优势胜过静态缓存。在其他提示词上记录的配置文件会从 4,500 个显存槽位中为 50% 的路由专家提供服务；换入对话自身的热门专家后，这一比例会提升到约 72%。

5. 在层内重叠两半并没有带来收益。把每个窗口拆分成两个 token 组，使一组的 CPU 专家运行，而 GPU 处理另一组，这种做法结果精确，但大约慢 7%（4K 下 85.8 对 91.8 token/秒）：稠密权重被读取两次，而且每组在其 token 之间共享的专家更少。

6. 最大的 CPU 收益来自指令组合。为 512 位 AVX-512 重写 Q2_0 expert 内核（每 64 个权重做一次 unpack、一次 VNNI 点积）使 CPU 的专家权重吞吐达到约 42 GB/s，接近 GPU 同时从 RAM 读取时 RAM 所能提供的水平（并发时 41 GB/s，单独时 52 GB/s）。

7. i-quants 受限于 CPU 算术运算，而不是 RAM。解码一个码本权重需要查表和符号处理；ggml 的 AVX2 内核和我们的 AVX-512 内核都约为每核 5 GB/s，六核时为 23–26 GB/s。对多个 token 只解码一次会有帮助（三个 token 时为 2.0–2.4×），但大多数专家只服务单个 token。ggml 的 CPU 后端在 x86 上完全没有针对 Q2_0 的 SIMD 内核；IQ 文件中的 Q2_0 down 投影需要它们自己的实现（比标量版本快 7×）。
8. 以字节而非槽位为单位确定缓存大小。IQ3_XXS 的专家根据层不同在 1.3 到 2.3 MB 之间。按各层而非最大专家来设定每个 VRAM 槽位的大小，可容纳 3,556 个专家而不是 2,673 个，并在 1K 下将解码速度提高 13%。
9. 带来回报的重叠：复制引擎。一部分未命中的专家可由 GPU 通过 PCIe 直接从 RAM 读取。若由复制内核完成，这些读取会处在 GPU 的关键路径上，并且超过 20% 的未命中会使解码变慢。改为在 CPU 线程规划该层时立即以 DMA 复制方式发出这些读取，可让 GPU 的复制引擎与 CPU 处理专家及 GPU 自身工作并行运行；GPU 先计算其已缓存的专家，待复制数据到达后再计算复制的专家。对于 i-quant 文件，其 CPU 受算术限制，并留出空闲的 RAM 带宽，现在 55% 的未命中通过 PCIe 传输：IQ2_XS 在 4K 下从每秒 70 个 token 提升到 78 个，IQ3_XXS 从 56 提升到 65。Q2_0 将复制内核保持在 20%：其 CPU 已经用上了 RAM 的带宽，因此 DMA 只会从它那里抢走带宽。

10. 在不让人等待的情况下重新填充缓存。自适应缓存过去会在轮次之间最多将 96 个专家（130–200 MB）复制到显存中，而下一轮会等待这些复制完成。立即驱逐旧专家，并且仅在新专家的副本到位后才接纳它，从而消除了等待：Q2_0 在 4K 下从每秒 91.7 个 token 提升到每秒 94.4 个 token。

11. 提示处理速度不随上下文长度变化。4K 下为每秒 539 个 token，262K 下为每秒 496 个 token：稀疏注意力只读取一组有界的位置。它受限于每个专家的内核启动和半精度反量化，仅使用 26 GB/s PCIe 链路中约 7 GB/s 的带宽。
12. 长上下文对显存的消耗大于对计算资源的消耗。在 262K token 时，8 位 KV 缓存只能为约 1,600 个专家留出空间，而不是 4,500 个；并且 MTP 层的草稿被接受的概率更低；二者共同使 Q2_0 解码速度从约每秒 95 个 token 降至约每秒 56 个 token。
13. 在 Windows 上使用固定内存需要小心。将 34–43 GB 作为一个范围进行页面锁定在 Windows 上会失败；按层划分范围，并为剩余部分加工作集锁是可行的，并且对未命中专家的 PCIe 读取绝不能跨越两个范围。

7 什么能让它更快

- 在请求之间保留对话状态。现在，每个请求都会重新处理其完整提示词。在每轮结束时保存循环状态（几百 MB）以及 KV 缓存，将会

turn a 30-second re-read of a long chat into a fraction of a second[5, 8]. For agents this is the largest single improvement available.

- **Overlap across layers, not within one.** Splitting each window into two token groups so that one group's CPU experts overlap the other group's GPU work is exact but slower (Finding 5); moving part of the misses to the copy engine is what paid (+10–16% for the i-quants). The next step in the same direction is to predict the next layer's experts from the current hidden state[15, 16] and start their copies before the router has run, which would widen the share the copy engine can take without the GPU waiting for it.
- **Cheaper i-quant arithmetic on the CPU.** Lookup-table kernels in the style of T-MAC[17], or converting the most-used experts to a CPU-friendly form at load time, would lift the i-quants toward the Q2_0 speed.
- **A faster prompt path.** Prompt speed is bound by per-expert kernel launches and half-precision dequantization, not by FLOPs; a fused 8-bit grouped kernel[8, 18] would help most on short and medium prompts.
- **Sampling.** Decoding is greedy today; temperature sampling needs rejection sampling in the verify window[11].
- **Hardware settings, free:** RAM at its EXPO speed, the monitor on the motherboard's output (it holds VRAM the expert cache could use), and Linux (lower driver overhead per launch than Windows' WDDM).

8 Other graphics cards (estimates)

We could only measure on the RTX 5070. Table 6 estimates two popular cards with the same CPU and 64 GB of DDR5, using the measured breakdown of every run: the GPU part is scaled by memory bandwidth, the CPU part by the share of experts that no longer fits (from the hit-rate curve of Figure 3 with the card's extra VRAM), and prompt speed by tensor throughput. The RTX 5060 Ti 16 GB has less bandwidth than the 5070 but more room for experts, so the two roughly cancel; the RTX 3090 has both more bandwidth and twice the VRAM, which removes most of the CPU work. Treat these as ±20% until someone measures them.

Card	Model	1K prompt / output	4K prompt / output	32K prompt / output	64K prompt / output	128K prompt / output	262K prompt / output
RTX 5060 Ti 16GB	Q2_0	~341 / ~80	~472 / ~87	~501 / ~81	~492 / ~72	~476 / ~62	~435 / ~53
RTX 5060 Ti 16GB	IQ2_XS	~291 / ~80	~406 / ~77	~434 / ~63	~426 / ~62	~413 / ~51	~383 / ~47
RTX 5060 Ti 16GB	IQ3_XXS	~249 / ~66	~359 / ~65	~381 / ~56	~374 / ~54	~363 / ~45	–
RTX 3090 24GB	Q2_0	~355 / ~128	~491 / ~140	~521 / ~130	~512 / ~115	~495 / ~100	~453 / ~85
RTX 3090 24GB	IQ2_XS	~303 / ~131	~422 / ~128	~451 / ~103	~444 / ~102	~430 / ~85	~398 / ~78
RTX 3090 24GB	IQ3_XXS	~260 / ~106	~374 / ~103	~396 / ~89	~390 / ~85	~378 / ~71	–

Table 6. Estimated tokens per second, same CPU (6-core AVX-512) and 64 GB DDR5. Not measured.

把对长对话的 30 秒重读缩短到零点几秒[5, 8]。对智能体来说，这是目前可用的最大单项改进。

- 跨层重叠，而不是在单层内重叠。将每个窗口拆分为两个 token 组，使一组在 CPU 上执行的专家计算与另一组的 GPU 工作重叠，虽然结果精确但速度更慢（发现 5）；将部分未命中迁移到复制引擎才真正见效（对 i-quants 为 +10–16%）。同一方向上的下一步，是根据当前隐藏状态[15, 16] 预测下一层的专家，并在路由器运行之前开始复制这些专家；这样一来，复制引擎可以承担更大份额，而无需 GPU 等待它。
- 在 CPU 上实现更廉价的 i-quant 运算。采用 T-MAC[17]，风格的查找表内核，或在加载时将最常用的专家转换为适合 CPU 的形式，都能让 i-quants 的速度向 Q2_0 速度靠拢。
- 更快的提示路径。提示速度受限于逐专家的内核启动和半精度反量化，而不是 FLOPs；融合的 8 位分组内核[8, 18] 对短提示和中等长度提示帮助最大。
- 采样。目前解码采用贪心策略；温度采样需要在验证窗口[11]中进行拒绝采样。
- 免费硬件设置：内存以 EXPO 速度运行，显示器接在主板输出上（它会占用专家缓存本可使用的 VRAM），以及 Linux（每次启动的驱动开销低于 Windows 的 WDDM）。

8 其他显卡（估算）

我们只能在 RTX 5070 上进行测量。表 6 对同一 CPU 和 64 GB DDR5 下的两款热门显卡进行了估算，使用了每次运行的实测分解：GPU 部分按内存带宽缩放，CPU 部分按不再放得下的专家所占比例缩放（根据图 3 中命中率曲线结合该显卡额外显存得出），提示处理速度按张量吞吐量缩放。RTX 5060 Ti 16 GB 的带宽低于 5070，但可容纳更多专家，因此二者大致抵消；RTX 3090 既有更高带宽，又有两倍显存，从而消除了大部分 CPU 工作。在有人实际测量之前，请将这些视为 ±20%。

Card	模型	1K 提示词 / 输出	4K 提示词 / 输出	32K 提示词 / 输出	64K 提示词 / 输出	128K 提示词 / 输出	262K 提示词 / 输出
RTX 5060 Ti 16GB	Q2_0	~341 / ~80	~472 / ~87	~501 / ~81	~492 / ~72	~476 / ~62	~435 / ~53
RTX 5060 Ti 16GB	IQ2_XS	~291 / ~80	~406 / ~77	~434 / ~63	~426 / ~62	~413 / ~51	~383 / ~47
RTX 5060 Ti 16GB	IQ3_XXS	~249 / ~66	~359 / ~65	~381 / ~56	~374 / ~54	~363 / ~45	–
RTX 3090 24GB	Q2_0	~355 / ~128	~491 / ~140	~521 / ~130	~512 / ~115	~495 / ~100	~453 / ~85
RTX 3090 24GB	IQ2_XS	~303 / ~131	~422 / ~128	~451 / ~103	~444 / ~102	~430 / ~85	~398 / ~78
RTX 3090 24GB	IQ3_XXS	~260 / ~106	~374 / ~103	~396 / ~89	~390 / ~85	~378 / ~71	–

表 6. 在相同 CPU（6 核 AVX-512）和 64 GB DDR5 下估算的每秒 token 数。未实测。

9 Related work

Computing mixture-of-experts layers partly on the CPU was explored by Eliseev and Mazur[22], Fiddler[13] and KTransformers[14]; MoE-Infinity[15] and Pre-gated MoE[16] study expert caching and next-layer prediction, and PowerInfer[23] exploits hot and cold activations on consumer GPUs. The idea of one engine for one model comes from Splash[5, 6], ninfer[7] and HyperQwen[8]. Speculative decoding is due to Leviathan et al.[11] and multi-token prediction heads to Gloeckle et al.[12]; lookup-table CPU kernels to T-MAC[17]; quantized GPU matrix kernels to Marlin[18]; paged attention to vLLM[19]. The i-quant formats, their dot products and the ggml CPU backend are llama.cpp's[9, 24].

References

1. Qwen Team. *On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability*. arXiv:2608.30320, Aug 2026.
2. Qwen Team. *Qwen3.8-Flash-Next: A New Architecture, Towards Ultimate Cost-Efficiency*. Blog post, Aug 2026. qwen.ai/blog?id=qwen3.8-flash-next
3. Qwen. Qwen3.8-Flash-Next model card, config.json and BF16 checkpoint. huggingface.co/Qwen/Qwen3.8-Flash-Next
4. ISTA-DASLab. Qwen3.8-Flash-Next-GSQ-RCO-GGUF model card, files and tensor-allocation/*.rco-allocation.txt. huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF
5. Inco AI. *Splash*. Blog post, 17 Sep 2026. inco.ai/blog/splash
6. Inco AI. splash (source repository, Apache-2.0). github.com/incoai/splash
7. Neroued. ninfer: README, docs/performance.md, docs/maintainer/engine-architecture.md, docs/maintainer/replayssm-gdn.md. github.com/Neroued/ninfer
8. syv.ai. HyperQwen: README, docs/optimizations.md, docs/benchmarks.md. github.com/syv-ai/HyperQwen
9. ggml-org. llama.cpp at commit 3cf0325: ggml/src/ggml-common.h (block formats, codebooks), ggml-cuda/vecdotq.cuh and dequantize.cuh (GPU dot products and dequantizers), ggml-cpu/quants.c (CPU dot products). MIT license. github.com/ggml-org/llama.cpp
10. S. Yang, J. Kautz, A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. 2024.
11. Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023.
12. F. Gloeckle et al. *Better & Faster Large Language Models via Multi-token Prediction*. ICML 2024.
13. K. Kamahori et al. *Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models*. 2024.
14. H. Chen et al. *KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models*. SOSP 2025.
15. L. Xue et al. *MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache*. 2024.
16. R. Hwang et al. *Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference*. ISCA 2024.
17. J. Wei et al. *T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge*. 2024.
18. E. Frantar et al. *MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models*. 2024.
19. W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023.
20. Z. Ye et al. *FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving*. MLSys 2025.
21. DeepSeek-AI. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*. 2025.
22. A. Eliseev, D. Mazur. *Fast Inference of Mixture-of-Experts Language Models with Offloading*. 2023.
23. Y. Song et al. *PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU*. 2023.
24. ggml-org. ggml CPU backend type traits (ggml_get_type_traits_cpu: vec_dot, from_float) at the same llama.cpp commit; NVIDIA. CUDA C++ Programming Guide (CUDA Graphs, mapped pinned memory), CUDA 13.

9 相关工作

Eliseev 和 Mazur[22]，、Fiddler[13] 和 KTransformers[14] 探索了将混合专家层部分放到 CPU 上计算；MoE-Infinity[15] 和 Pre-gated MoE[16] 研究了专家缓存与下一层预测，PowerInfer[23] 则利用消费级 GPU 上的热激活和冷激活。一个模型对应一个引擎的想法来自 Splash[5, 6]，、ninfer[7] 和 HyperQwen[8]。推测解码由 Leviathan 等人[11] 提出，多 token 预测头由 Gloeckle 等人[12]提出；查表式 CPU 内核来自 T-MAC[17]；量化 GPU 矩阵内核来自 Marlin[18]；分页注意力来自 vLLM[19]。i-quant 格式、其点积以及 ggml CPU 后端都是 llama.cpp 的[9, 24]。

参考文献

1. Qwen 团队。关于 Qwen3.8-Next 架构的设计：评估、效率与训练稳定性。arXiv:2608.30320，2026 年 8 月。2. Qwen 团队。Qwen3.8-Flash-Next：一种新架构，迈向极致成本效率。博客文章，2026 年 8 月。qwen.ai/blog?id=qwen3.8-flash-next 3. Qwen。Qwen3.8-Flash-Next 模型卡、config.json 和 BF16 检查点。huggingface.co/Qwen/Qwen3.8-Flash-Next 4. ISTA-DASLab。Qwen3.8-Flash-Next-GSQ-RCO-GGUF 模型卡、文件以及 tensor-allocation/*.rco-allocation.txt。huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF 5. Inco AI。Splash。博客文章，2026 年 9 月 17 日。inco.ai/blog/splash 6. Inco AI。splash（源代码仓库，Apache-2.0）。github.com/incoai/splash 7. Neroued。ninfer：README、docs/performance.md、docs/maintainer/engine-architecture.md、docs/maintainer/replayssm- gdn.md。github.com/Neroued/ninfer 8. syv.ai。HyperQwen：README、docs/optimizations.md、docs/benchmarks.md。github.com/syv-ai/HyperQwen 9. ggml-org。提交 3cf0325 的 llama.cpp：ggml/src/ggml-common.h（块格式、码本）、ggml-cuda/vecdotq.cuh 和 dequantize.cuh（GPU 点积与反量化器）、ggml-cpu/quants.c（CPU 点积）。MIT 许可证。github.com/ggml-org/llama.cpp 10. S. Yang, J. Kautz, A. Hatamizadeh。门控 Delta 网络：用 Delta 规则改进 Mamba2。2024。11. Y. Leviathan, M. Kalman, Y. Matias。通过推测解码实现 Transformer 的快速推理。ICML 2023。12. F. Gloeckle 等。通过多 token 预测实现更优且更快的大语言模型。ICML 2024。13. K. Kamahori 等。Fiddler：面向混合专家模型快速推理的 CPU-GPU 编排。2024。14. H. Chen 等。KTransformers：释放 MoE 模型 CPU/GPU 混合推理的全部潜力。SOSP 2025。15. L. Xue 等。MoE-Infinity：在个人机器上利用稀疏感知专家缓存实现高效 MoE 推理。2024。16. R. Hwang 等。预门控 MoE：面向快速且可扩展的混合专家推理的算法-系统协同设计。ISCA 2024。17. J. Wei 等。T-MAC：通过查表实现 CPU 复兴，推动低比特 LLM 在边缘部署。2024。18. E. Frantar 等。MARLIN：大语言模型上的混合精度自回归并行推理。2024。19. W. Kwon 等。基于 PagedAttention 的大语言模型服务高效内存管理。SOSP 2023。20. Z. Ye 等。FlashInfer：面向 LLM 推理服务的高效且可定制的注意力引擎。MLSys 2025。21. DeepSeek-AI。DeepSeek-V3.2：推动开放大语言模型的前沿。2025。22. A. Eliseev, D. Mazur。通过卸载实现混合专家语言模型的快速推理。2023。23. Y. Song 等。PowerInfer：使用消费级 GPU 实现快速大语言模型服务。2023。24. ggml-org。同一 llama.cpp 提交中的 ggml CPU 后端类型特征（ggml_get_type_traits_cpu: vec_dot, from_float）；NVIDIA。CUDA C++ 编程指南（CUDA Graphs、映射固定内存），CUDA 13。